[Համալսարանի անվանում]

[Ֆակուլտետ]

Միջամբիոնային դիպլոմային նախագծի առաջադրանք

FPGA-ի հիման վրա սենսորային տվյալների հավաքագրման, մշակման և վերլուծության ինտեգրված ապարատածրագրային հարթակի մշակում

FPGA-Based Sensor Data Acquisition, Processing and Analytics Platform

---

Ուսանող 1: կնշանակվի
Միկրոէլեկտրոնիկայի ամբիոն

Ուսանող 2: կնշանակվի
Ծրագրային / Համակարգչային ճարտարագիտության ամբիոն

Ղեկավար 1: [Ղեկավարի անուն, ազգանուն, աստիճան]

Ղեկավար 2: [Ղեկավարի անուն, ազգանուն, աստիճան]

Երևան, 2026

Տարբերակ 1.0 · Օրինակ

Նախագիծը մեկ հայացքով

FPGA-ի հիման վրա սենսորային տվյալների հավաքագրման, մշակման և վերլուծության ինտեգրված ապարատածրագրային հարթակի մշակում

FPGA-Based Sensor Data Acquisition, Processing and Analytics Platform

Աշխատանքի տեսակ	Կրթական աստիճան	Չևաչափ	Տևողություն	Տեղերի քանակ
Բակալավրի դիպլոմային նախագիծ	**Բակալավր**	**Միջամբիոնային (երկու ուսանող)**	**2 կիսամյակ (մոտ 30 շաբաթ)**	**2 (մեկական յուրաքանչյուր ամբիոնից)**

Մասնակիցներ

Ուսանող 1: կնշանակվի
Միկրոէլեկտրոնիկայի ամբիոն
Ղեկավար: [Ղեկավարի անուն, ազգանուն, աստիճան]

Ուսանող 2: կնշանակվի
Ծրագրային / Համակարգչային ճարտարագիտության ամբիոն
Ղեկավար: [Ղեկավարի անուն, ազգանուն, աստիճան]

Ժամկետներ

Մեկնարկ
Սեպտեմբեր, 2026

Դիմելու վերջնաժամկետ
[ամսաթիվ]

Աշխատանքի լեզու
Հայերեն

Ոլորտ և պահանջներ

Ոլորտներ	Թվային նախագծում · FPGA · ներդրված համակարգեր · Web հարթակներ · տվյալների վերլուծություն
Բանալի բառեր	FPGA · Zynq UltraScale+ · AXI · RTL · սենսոր-էմուլյացիա · runtime կոնֆիգուրացիա · տվյալների վերլուծություն
Գործիքներ և սարքավորումներ	AMD AUP ZU3 (Zynq UltraScale+ MPSoC), Vivado, Vitis; սերվերային համակարգիչ, Python, SQL տվյալների բազա
Նախնական պահանջներ	Թվային սխեմաներ և HDL (ուսանող 1); ծրագրավորում, տվյալների բազաներ, Web (ուսանող 2)

Բովանդակություն

1 Նախագծի համատեքստ . . . 1
2 Խնդիրը և նախագծի գաղափարը . . . 1
3 Նպատակներ և սպասվող արդյունքներ . . . 3
4 Շրջանակ, MVP և սահմանափակումներ . . . 4
5 FPGA տվյալների գեներացիա և մշակում . . . 5
6 PS–PL հաղորդակցություն և runtime կոնֆիգուրացիա . . . 6
7 Տվյալների ֆորմատ և մոդել . . . 7
8 Դերեր և խնդիրներ . . . 7
9 Ուսումնական ուղի . . . 9
10 Աշխատանքի մեկնարկ . . . 11

11 Աշխատանքային պլան և հանգրվաններ 13
12 Արդյունքներ 13
13 Գնահատում և ընդունման չափանիշներ 14
14 Ամփոփ ցուցադրման սցենար 15
15 Ընդլայնումներ (ընտրովի) 16
16 Ղեկավարում, համագործակցություն և ռիսկեր 16
Գրականություն և ընթերցանության ցանկ 17
Հապավումների ցանկ 18
Բառարան 18
Հաստատում 19

1. Նախագծի համատեքստ

Սույն առաջադրանքը սահմանում է **միջամբիոնային բակալավրիատի դիպլոմային նախագիծ**, որը կատարվում է **երկու ուսանողի** կողմից համատեղ.

- **Ուսանող 1 (Միկրոէլեկտրոնիկա)** — թվային նախագծման / FPGA բաղադրիչ:
- **Ուսանող 2 (Ծրագրային / համակարգչային ճարտարագիտություն, Ավտոմատացում, Ալգորիթմներ կամ հարակից ամբիոն)** — ծրագրային ապահովման, տվյալների վերլուծության, վիզուալիզացիայի և մոդելավորման բաղադրիչ:

Ապարատային հենքը. FPGA հարթակ՝ **AMD AUP ZU3**; սարքերի ընտանիք՝ **AMD Zynq UltraScale+ MPSoC**; նախագծման գործիքներ՝ **AMD Vivado** և **Vitis**:

Նախագիծը պետք է պարունակի էական թվային նախագծման / FPGA բաղադրիչ առաջին ուսանողի համար և միևնույն ժամանակ ապահովի **անկախ** ծրագրային, տվյալների վերլուծության, վիզուալիզացիայի ու մոդելավորման խնդիր երկրորդ ուսանողի համար: Համակարգը պետք է աջակցի **երկկողմանի հաղորդակցություն**.

1. **FPGA → հարթակ.** հավաքագրման և մշակման արդյունքների փոխանցում,
2. **հարթակ → FPGA.** աշխատանքային ռեժիմում (runtime) կոնֆիգուրացիա և կառավարում:

2. Խնդիրը և նախագծի գաղափարը

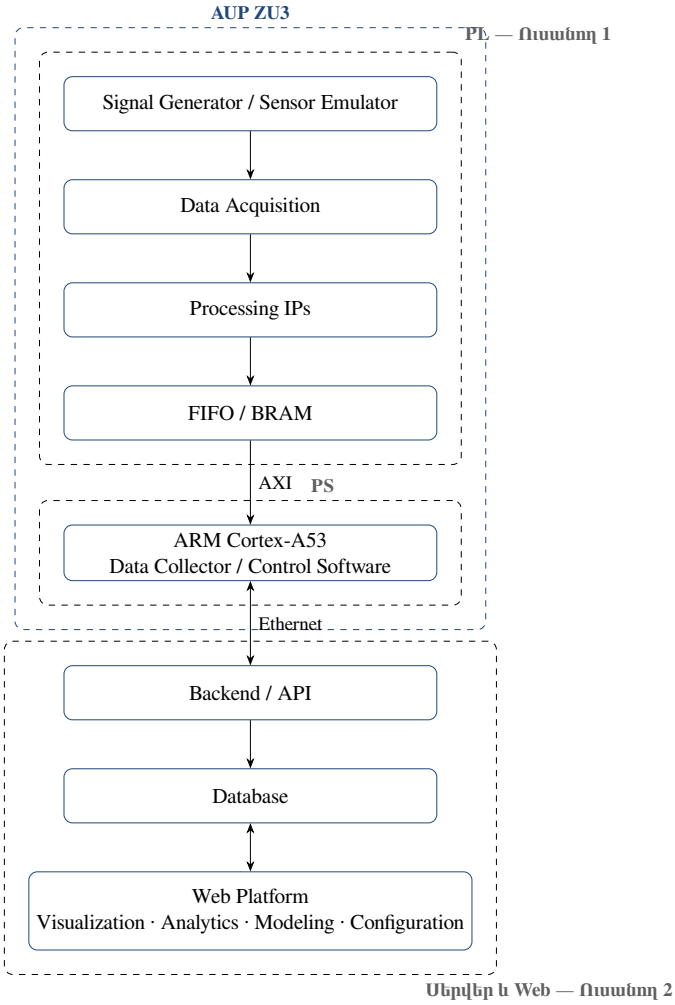
Նպատակն է մշակել ինտեգրված ապարատա-ծրագրային հարթակ, որտեղ FPGA-ն գեներացնում կամ ստանում է սենսորանման ազդանշաններ, կատարում է իրական ժամանակի նախնական մշակում (preprocessing) և ստացված տվյալները փոխանցում սերվերին: Web հարթակը պահպանում, վիզուալիզացնում, վերլուծում և մոդելավորում է հավաքագրված տվյալները, ինչպես նաև աշխատանքային ռեժիմում կոնֆիգուրացնում է FPGA ապարատը:

Խնդրի ձևակերպում

Անհրաժեշտ է կառուցել *Web հարթակ* ↔ *Server* ↔ *Zynq PS* ↔ *FPGA* ամբողջական համակարգ, որում FPGA-ն իրական ժամանակում գեներացնում ու մշակում է սենսորային տվյալները, իսկ ծրագրային հարթակը դրանք պահպանում, վերլուծում է և հետադարձ կապով փոխում FPGA-ի պարամետրերը՝ առանց նոր bitstream գեներացնելու:

Սկզբնական իրականացումը **ֆիզիկական սենսորներ չի պահանջում**. սենսորի վարքը էմուլացվում է անմիջապես FPGA-ի ներսում՝ կոնֆիգուրելի RTL ազդանշան-գեներատոր մոդուլներով: Ֆիզիկական սենսորները կարող են միանալ հետագայում՝ որպես ընդլայնում:

Ինչպես ցույց է տրված Նկ. 2.1-ում, FPGA-ի ծրագրավորվող տրամաբանությունը (PL) գեներացնում և մշակում է ազդանշանները, ապա **AXI**-ի միջոցով տվյալները փոխանցվում են մշակող համակարգին



Նկար 2.1: Համակարգի ընդհանուր ճարտարապետությունը՝ AUP ZU3-ի PL/PS հատվածներից մինչև Web հարթակ.

(PS՝ ARM Cortex-A53): PS-ը **Ethernet**-ով կապվում է սերվերի հետ, որտեղ գործում են Backend/API-ն, տվյալների բազան և Web հարթակը (վիզուալիզացիա, վերլուծություն, մոդելավորում, կոնֆիգուրացիա):

3. Նպատակներ և սպասվող արդյունքներ

3.1 Նախագծի նպատակները

1. FPGA-ի ներսում իրականացնել կոնֆիգուրելի սենսոր-էմուլյացիա (RTL ազդանշան-գեներատորներ) առնվազն 3 սենսորի տիպի համար:
2. Իրականացնել իրական ժամանակի նախնական մշակում (առնվազն 3 processing IP) FPGA-ի վրա:
3. Ապահովել PL→PS տվյալների փոխանցում և PS→սերվեր հաղորդակցություն:
4. Մշակել սերվեր, տվյալների բազա և Web հարթակ՝ պահպանման, վիզուալիզացիայի և վերլուծության համար:
5. Ապահովել FPGA-ի runtime կոնֆիգուրացիա Web հարթակից՝ AXI4-Lite ռեգիստրների միջոցով:
6. Ցուցադրել համակարգի ամբողջական աշխատանքը՝ փորձի (experiment (փորձ)) կազմակերպման, տվյալների հավաքագրման և վերլուծության միջոցով:

3.2 Ուսումնական արդյունքներ

Նախագծի ավարտին ուսանողները կկարողանան.

- **Ուսանող 1.** նախագծել և վերիֆիկացնել RTL մոդուլներ testbench-երով, ինտեգրել դրանք AXI-ով Zynq PS-ի հետ, վերլուծել timing-ը և ռեսուրսների օգտագործումը:
- **Ուսանող 2.** նախագծել տվյալների մոդել և API, կառուցել Web dashboard իրական ժամանակի տվյալներով, կիրառել վիճակագրական վերլուծություն և անոմալիաների հայտնաբերում:
- **Երկուսն էլ.** համաձայնեցնել միջերեսներ, աշխատել git-ով, ինտեգրել և ցուցադրել ամբողջական համակարգ:

4. Շրջանակ, MVP և սահմանափակումներ

Բակալավրիատի ծավալը իրատեսական պահելու համար առաջին տարբերակը (Minimum Viable Product (MVP)) պետք է պարունակի.

4.1 MVP — ապարատային մաս

- մեկ AUP ZU3 հարթակ,
- 3 էմուլացված սենսորի տիպ՝ Temperature, Pressure, Vibration,
- վերաօգտագործելի գեներատորներ՝ Ramp, Periodic, LFSR/Noise, Anomaly injection,
- FIFO/BRAM բուֆերացում,
- մոտ 3 processing IP՝ Moving Average, Min/Max, Threshold/Event Detection,
- AXI4-Lite կոնֆիգուրացիա, PL→PS տվյալների փոխանցում, Ethernet հաղորդակցություն:

4.2 MVP — ծրագրային մաս

- մեկ սերվերային համակարգիչ (Backend/API, տվյալների բազա),
- Web dashboard՝ փորձի կոնֆիգուրացիայով,
- տվյալների վիզուալիզացիա (live և պատմական), փորձերի համեմատում,
- հիմնական վիճակագրական վերլուծություն,
- մեկ մոդելավորման / անոմալիաների հայտնաբերման կիրառություն:

4.3 Շրջանակից դուրս

- ֆիզիկական սենսորներ, AXI DMA, FPGA reconfiguration, Linux PS-ի վրա, closed-loop կառավարում — տե՛ս ընդլայնումները (բաժին 15):
- արտադրական մակարդակի անվտանգություն և օգտատերերի կառավարում Web հարթակում:

4.4 Սահմանափակումներ և ենթադրություններ

- Հասանելի է մեկ AUP ZU3 հարթակ. ուսանող 2-ը մինչև ինտեգրումը աշխատում է էմուլյատորով կամ գրանցված տվյալներով (տե՛ս բաժին 8):
- AUP ZU3-ի սարքը (XCZU3EG) աջակցվում է Vivado/Vitis-ի անվճար (Standard) տարբերակով:

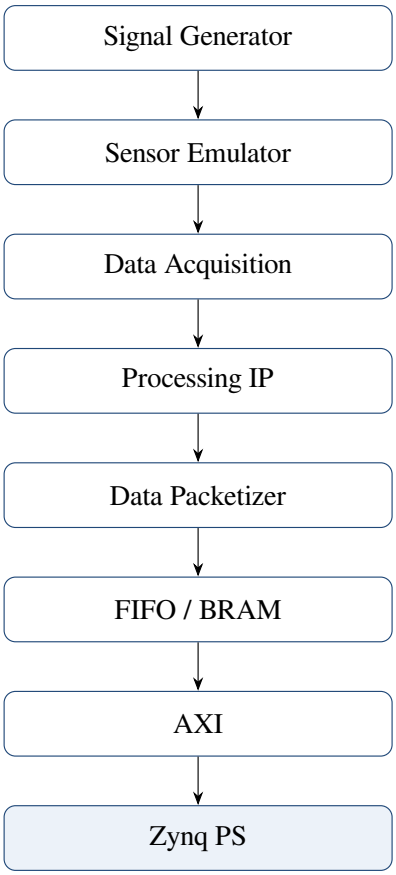
- Սերվերային ծրագրային ապահովման տեխնոլոգիաները ուսանող 2-ն ընտրում է ղեկավարի հետ համաձայնեցմամբ. բաժին 9-ում բերված են առաջարկվող տարբերակները:

5. FPGA տվյալների գեներացիա և մշակում

Սկզբնական տարբերակում իրական սենսորները պարտադիր չեն: Միկրոէլեկտրոնիկայի ուսանողը պետք է իրականացնի **կոնֆիգուրելի ազդանշան-գեներատոր / սենսոր-էմուլյատոր** RTL մոդուլներ, որոնք FPGA-ի ներսում մոդելավորում են սենսորի իրատեսական վարքը: Հնարավոր սենսորների տիպերը՝ ջերմաստիճան (Temperature), ճնշում (Pressure), վիբրացիա (Vibration), ընդհանրական անալոգային սենսոր և այլ կոնֆիգուրելի մոդելներ:

Յուրաքանչյուր սենսորի համար առանձին ապարատ իրականացնելու փոխարեն նախատեսվում են **վերաօգտագործելի ազդանշան-գեներացման պրիմիտիվներ**.

- **Ramp Generator** — աստիճանաբար աճող/նվազող արժեքներ (ջերմաստիճան, ճնշում):
- **Periodic Signal Generator** — պարբերական ազդանշաններ (վիբրացիա):
- **Pseudo-Random / Noise Generator** — Linear-Feedback Shift Register (LFSR)-ի հիման վրա, կոնֆիգուրելի աղմուկ:
- **Anomaly / Fault Generator** — ոչ նորմալ վարքի ներարկում (գերտաքացում, բարձր վիբրացիա):



Նկար 5.1: FPGA-ի RTL մշակման հնարավոր փոխակրիչը (pipeline).

Signal Generator-ի կոնֆիգուրելի պարամետրերն են՝ ազդանշանի տիպ, ամպլիտուդ, հաճախություն, բազային արժեք, նմուշառման արագություն, աղմուկի մակարդակ, անոմալիայի միացում և մեծություն: **Sensor Emulator**-ը ընդհանրական ազդանշանը վերածում է սենսորանման արժեքների՝ scaling, offset, saturation, աղմուկի ու անոմալիայի ներարկման միջոցով:

FPGA-ն կատարում է իրական ժամանակի մշակում (**Processing IP**-ներ). հնարավոր մոդուլներ՝ Moving Average, Min/Max Detector, Threshold Detector, Event Detector, Finite Impulse Response (FIR) Filter, Statistical accumulator: MVP-ն պարունակում է **3 մշակող մոդուլ**. որոնք ընտրել՝ որոշում է ուսանող 1-ը ղեկավարի հետ:

6. PS–PL հաղորդակցություն և runtime կոնֆիգուրացիա

Zynq-ի մշակող համակարգը (PS) ծրագրավորվող տրամաբանության (PL) հետ կապվում է **AXI**-ի միջոցով՝ երկու ուղիներով.

- **Data Path** (AXI-Stream / FIFO) — տվյալների փոխանցում PL→PS:
- **Control Path** (AXI4-Lite) — FPGA մոդուլների կոնֆիգուրացիա՝ ռեգիստրների միջոցով:

Web հարթակը պետք է կարողանա փոխել FPGA-ի վարքը **առանց bitstream վերագեներացնելու**՝ AXI4-Lite ռեգիստրների միջոցով (Աղյուսակ 6.1):

Աղյուսակ 6.1: AXI4-Lite կոնֆիգուրացիոն ռեգիստրների քարտեզի օրինակ.

Հասցե	Ռեգիստր	Հասցե	Ռեգիստր
0x00	CONTROL	0x18	NOISE_LEVEL
0x04	SENSOR_TYPE	0x1C	THRESHOLD
0x08	SAMPLE_RATE	0x20	FILTER_ENABLE
0x0C	SIGNAL_TYPE	0x24	FILTER_WINDOW
0x10	AMPLITUDE	0x28	ANOMALY_ENABLE
0x14	FREQUENCY	0x2C	ANOMALY_LEVEL

Կոնֆիգուրացիայի ուղին է՝ Web Platform → Backend/API → Ethernet → Zynq PS → AXI4-Lite → FPGA ռեգիստրներ → RTL մոդուլներ:

Նշում

Պետք է հստակ տարբերել **runtime կոնֆիգուրացիան** (արդեն իրականացված ապարատի պարամետրերի փոփոխություն՝ պարտադիր) և **FPGA reconfiguration**-ը (նախապես գեներացված bitstream-ների միջև ընտրություն՝ ընդլայնում, ոչ MVP):

Բուֆերացումն իրականացվում է FIFO/BRAM-ով՝ Processing IP → AXI FIFO → Zynq PS: Բարձր տվյալահոսքի դեպքում AXI4-Stream → AXI DMA → DDR ուղին դիտվում է որպես ընդլայնում:

7. Տվյալների ֆորմատ և մոդել

FPGA/PS տվյալները սերվեր ուղարկելուց առաջ ձևափոխվում են կառուցվածքային ֆորմատի.

```
{ "device": "AUP_ZU3_01", "experiment_id": 12, "sensor": "vibration",
  "timestamp": 1789063200, "sequence": 1524,
  "raw_value": 728, "processed_value": 681, "status": "normal" }
```

PS-ը տվյալները ուղարկում է API-ին, օրինակ՝ `POST /api/v1/data`:

Տվյալների բազան պետք է աջակցի **փորձերի (experiments)** հասկացությունը.

- **Experiment** — `experiment_id`, `device_id`, `configuration`, `start_time`, `end_time`:
- **Sensor** — `sensor_id`, `sensor_type`, `sample_rate`, `unit`:
- **Sample** — `experiment_id`, `sensor_id`, `timestamp`, `sequence_number`, `raw_value`, `processed_value`, `status`:

Այս մոդելը թույլ է տալիս համեմատել փորձերը (օր.՝ Experiment #12 vs #18):

8. Դերեր և խնդիրներ

8.1 Ուսանող 1 — Միկրոէլեկտրոնիկա

Անհատական թեմա. *AUP ZU3 FPGA-ի հիման վրա սենսորային տվյալների գեներացման, հավաքագրման և նախնական մշակման ապարատային համակարգի նախագծում.*

Խ1.1 Նախագծել կոնֆիգուրելի RTL ազդանշան-գեներատորներ (Ramp, Periodic):

Խ1.2 Իրականացնել սենսոր-էմուլյացիայի տրամաբանությունը (scaling, offset, saturation):

Խ1.3 Իրականացնել LFSR-ի վրա հիմնված աղմուկի գեներատոր:

Խ1.4 Իրականացնել անոմալիաների ներարկում:

Խ1.5 Մշակել տվյալների հավաքագրման և packetizer տրամաբանությունը:

Խ1.6 Իրականացնել FIFO/BRAM բուֆերացում:

Խ1.7 Մշակել 3 կոնֆիգուրելի processing IP:

Խ1.8 Իրականացնել AXI4-Lite կոնֆիգուրացիոն ռեգիստրները (Աղյուսակ 6.1):

Խ1.9 Իրականացնել PL→PS տվյալների փոխանցումը AXI-ով և ինտեգրել PL-ը Zynq PS-ի հետ:

Խ1.10 Գրել PS-ի ծրագիրը (Vitis)՝ տվյալները Ethernet-ով ուղարկելու և հրամաններ ընդունելու համար:

Խ1.11 Մշակել testbench-եր յուրաքանչյուր մոդուլի համար:

Խ1.12 Կատարել սինթեզ, implementation, timing analysis և LUT/FF/DSP/BRAM օգտագործման վերլուծություն:

Խ1.13 Վալիդացնել ամբողջական նախագիծը AUP ZU3-ի վրա:

Խ1.14 (ընտրովի) Իրականացնել FIR ֆիլտր կամ AXI DMA ուղի:

Կարևոր

Ուսանող 1-ի դիպլոմը կենտրոնանում է թվային նախագծման, RTL-ի, FPGA ճարտարապետության, data/control ուղիների, բուֆերացման, AXI-ի, վերիֆիկացիայի, սինթեզի, timing-ի և ապարատային վալիդացիայի վրա:

8.2 Ուսանող 2 — Ծրագրային ապահովում / Ալգորիթմներ

Անհատական թեմա. *FPGA-ից ստացվող սենսորային տվյալների պահպանման, վիզուալիզացիայի և վերլուծության ծրագրային հարթակի մշակում.*

Խ2.1 Նախագծել տվյալների բազայի մոդելը (Experiment, Sensor, Sample):

Խ2.2 Մշակել Backend/API՝ տվյալների ընդունման և կոնֆիգուրացիոն հրամանների համար:

Խ2.3 Գրել **FPGA էմուլյատոր** (սկրիպտ), որը ուղարկում է նույն ֆորմատի տվյալներ, որպեսզի աշխատանքը չկախվի ապարատից:

Խ2.4 Մշակել Web հավելված՝ փորձի կոնֆիգուրացիայի ինտերֆեյսով:

Խ2.5 Իրականացնել live և պատմական տվյալների վիզուալիզացիա և փորձերի համեմատում:

Խ2.6 Իրականացնել վիճակագրական վերլուծություն (mean, variance, std, histogram):

Խ2.7 Իրականացնել անոմալիաների հայտնաբերման մեկ մեթոդ և համեմատել այն FPGA-ի threshold detector-ի հետ:

Խ2.8 (ընտրովի) Իրականացնել կանխատեսում (prediction) կամ FPGA-ի պարամետրերի ավտոմատ փոփոխում:

Կարևոր

Ուսանող 2-ի դիպլոմը կենտրոնանում է տվյալների բազայի, Backend-ի, Web հարթակի, վիզուալիզացիայի, վիճակագրության և վերլուծության/մոդելավորման վրա և անկախ է RTL-ի մշակումից:

8.3 Ընդհանուր խնդիրներ և միջերեսներ

Խ0.1 Մինչև 5-րդ շաբաթը համաձայնեցնել և փաստաթղթավորել միջերեսները (Աղյուսակ 8.1):

Խ0.2 Կազմակերպել ընդհանուր git ռեպոզիտորիա և ինտեգրման թեստեր:

Խ0.3 Իրականացնել ամբողջական ինտեգրում և համատեղ ցուցադրում (բաժին 14):

Աղյուսակ 8.1: Միջերեսները երկու մասերի միջև.

Միջերես	Ուղղություն	Համաձայնեցված ձևաչափ
Տվյալների հոսք	PS → սերվեր	JSON (բաժին 7), `POST /api/v1/data`, Ethernet
Կոնֆիգուրացիա	սերվեր → PS	JSON հրաման՝ ռեգիստրի անուն և արժեք (Աղյուսակ 6.1)
Փորձի կառավարում	սերվեր → PS	`start` / `stop` հրամաններ, `experiment_id`

9. Ուսումնական ուղի

9.1 Նախնական պահանջներ

Ուսանող 1

- Թվային սխեմաներ (կոմբինացիոն/հաջորդական տրամաբանություն, FSM) — *ինքնաստուգում.* կարող է նախագծել counter և FSM, գծել ժամանակային դիագրամ:
- Verilog/SystemVerilog կամ VHDL հիմունքներ — *ինքնաստուգում.* գրել և սիմուլացնել պարզ մոդուլ: Եթե թույլ է՝ [B1], գլ. 4:
- C ծրագրավորում — PS-ի ծրագրի համար:

Ուսանող 2

- Ծրագրավորում (Python կամ JavaScript/TypeScript) — *ինքնաստուգում.* գրել փոքր REST սերվեր:
- Տվյալների բազաներ և SQL — *ինքնաստուգում.* նախագծել 3 աղյուսակով սխեմա և գրել JOIN հարցում:
- Հավանականություն և վիճակագրություն (mean, variance, բաշխումներ):

9.2 Ինչ սովորել

Ծանոթություն — գիտի՝ ինչ է և երբ է կիրառվում. **Ընբռնում** — կարող է բացատրել և հասկանալ առկա աշխատանքը. **Կիրառում** — կարող է ինքնուրույն կիրառել:

Աղյուսակ 9.1: Գիտելիքների քարտեզ — Ուսանող 1.

Թեմա	Մակարդակ	Որտեղ է կիրառվում նախագծում	Աղբյուրներ
Թվային նախագծում և վերիֆիկացիա			
Սինքրոն նախագծում, FSM, pipeline	Կիրառում	գեներատորներ, processing IP	[B1], գլ. 3–4
Testbench և սիմուլյացիա	Կիրառում	բոլոր մոդուլների վերիֆիկացիա	[U1]; [B2]
LFSR, ֆիքսված ստորակետով թվաբանություն	Կիրառում	աղմուկ, scaling, ֆիլտրեր	[B2]
Moving average և FIR ֆիլտրեր	Ընթռնում	processing IP	[B3], գլ. 14–15
Zynq և AXI			
Zynq UltraScale+ ճարտարապետություն (PS/PL)	Ընթռնում	համակարգի ինտեգրում	[U2]; [B4]
AXI4-Lite slave	Կիրառում	կոնֆիգուրացիոն ռեգիստրներ	[U3]; [B4]
AXI4-Stream և AXI FIFO	Կիրառում	տվյալների ուղի PL→PS	[U4]
AXI DMA, DDR	Ծանոթություն	ընդլայնում	[U2]
Իրականացում			
Vivado block design, IP packaging	Կիրառում	PL-ի ինտեգրում	[B4]
Timing constraints և timing closure	Ընթռնում	implementation, հաշվետվություն	[U5]
Vitis bare-metal ծրագիր, Ethernet (lwIP)	Ընթռնում	PS-ի ծրագիր	**[AMD-ի lwIP օրինակ — ստուգել]**

Աղյուսակ 9.2: Գիտելիքների քարտեզ — Ուսանող 2.

Թեմա	Մակարդակ	Որտեղ է կիրառվում նախագծում	Աղբյուրներ
Backend և տվյալներ			
REST API նախագծում	Կիրառում	տվյալների ընդունում, հրամաններ	[W1]
Ռելյացիոն մոդելավորում, ժամանակային շարքեր	Կիրառում	Experiment/Sample մոդել	[W2]; [B5], գլ. 2–3

Թեմա	Մակարդակ	Որտեղ է կիրառվում նախագծում	Աղբյուրներ
Հոսքային տվյալներ (WebSocket կամ polling)	Ընթռնում	live վիզուալիզացիա	[B5], գլ. 11
Վերլուծություն			
Նկարագրական վիճակագրություն	Կիրառում	փորձերի համեմատում	[B6]
Անոմալիաների հայտնաբերում	Կիրառում	մոդելավորման բաղադրիչ	[A1]
Ազդանշանի ֆիլտրում	Ընթռնում	FPGA և սերվերի մշակման համեմատություն	[B3], գլ. 15
Համակարգ			
Zynq PS/PL-ի բաժանում, AXI ռեգիստրներ	Ծանոթություն	կոնֆիգուրացիոն հրամանների իմաստ	[B4]

9.3 Գործիքներ և հմտություններ

- **Vivado / Vitis** (ուսանող 1) — սինթեզ, implementation, PS-ի ծրագիր:
- **Python (FastAPI), PostgreSQL, Web frontend** (ուսանող 2) — առաջարկվող տարբերակ. վերջնական ընտրությունը համաձայնեցվում է ղեկավարի հետ:
- **git** (երկուսն էլ) — ընդհանուր ռեպոզիտորիա, branch-եր, code review:

10. Աշխատանքի մեկնարկ

10.1 Առաջին հանդիպման ստուգաթերթ

- Ընդհանուր git ռեպոզիտորիա և համատեղ թղթապանակ:
- AUP ZU3 հարթակի հանձնում (ուսանող 1), լաբորատորիայի մուտք և անվտանգության հրահանգավորում:
- Սերվերային համակարգչի կամ վիրտուալ մեքենայի հասանելիություն (ուսանող 2):
- Հաղորդակցման ալիք և շաբաթական հանդիպման ժամ:

10.2 Միջավայրի պատրաստում

1. **Ուսանող 1.** տեղադրել Vivado և Vitis (նույն տարբերակը, որ նշված է ռեպոզիտորիայում), ավելացնել AUP ZU3-ի board files-ը, կառուցել և բեռնել «hello world» նախագիծ:

2. **Ուսանող 2.** տեղադրել Python, PostgreSQL և ընտրված frontend գործիքները, գործարկել դատարկ API և dashboard:

10.3 Ներածական վարժություններ

Ներածական վարժություն 1: Counter և testbench (ուսանող 1) Մոտ. ժամանակ՝ 6 ժ

Նախագծել կոնֆիգուրելի counter, որը պարամետրով ընտրում է քայլը, և գրել ինքնաստուգվող testbench:

Կատարված է, երբ՝ սիմուլյացիան բոլոր դեպքերում PASS է ցույց տալիս, կոդը ռեպոզիտորիայում է:

Ներածական վարժություն 2: LED՝ AXI4-Lite ռեգիստրով (ուսանող 1) Մոտ. ժամանակ՝ 10 ժ

Ստեղծել AXI4-Lite IP մեկ ռեգիստրով, որը կառավարում է LED-ի թարթման հաճախությունը, և փոխել այն PS-ի ծրագրից:

Կատարված է, երբ՝ ռեգիստրում գրված արժեքը փոխում է թարթման հաճախությունը հարթակի վրա:

Ներածական վարժություն 3: Տվյալների ընդունում և գրաֆիկ (ուսանող 2) Մոտ. ժամանակ՝ 8 ժ

Գրել API endpoint, որը ընդունում է բաժին 7-ի JSON-ը, պահում է բազայում, և սկրիպտ, որը վայրկյանում 10 սինուսոիդային նմուշ է ուղարկում:

Կատարված է, երբ՝ dashboard-ը իրական ժամանակում ցույց է տալիս ազդանշանը, տվյալները պահված են բազայում:

Ներածական վարժություն 4: Միջերեսի համաձայնեցում (երկուսն էլ) Մոտ. ժամանակ՝ 4 ժ

Համատեղ լրացնել Աղյուսակ 8.1-ը և գրել JSON հաղորդագրությունների օրինակներ ռեպոզիտորիայում:

Կատարված է, երբ՝ ղեկավարները հաստատել են միջերեսի փաստաթուղթը:

10.4 Առաջին ընթերցանություն

Ուսանող 1. [B4] (PS/PL և AXI գլուխները), ապա [U3]-ի AXI4-Lite մասը: **Ուսանող 2.** [W1] (Tutorial), [B5], գլ. 2:

11. Աշխատանքային պլան և հանգրվաններ

Աղյուսակ 11.1: Աշխատանքային պլան (մոտ 30 շաբաթ).

Շաբաթներ	Փուլ / հանգրվան	Ստուգման արդյունք
1–3	Մեկնարկ. միջավայր, ընթերցանություն, ներածական վարժություններ	Վարժությունները ընդունված են
4–5	Միջերեսների համաձայնեցում, ճարտարապետության մանրամասնում	Միջերեսի փաստաթուղթ, ռեգիստրների քարտեզ
6–12	Ու1. գեներատորներ, էմուլյատոր, FIFO + testbench-եր. Ու2. բազա, API, էմուլյատոր-սկրիպտ, dashboard	Սիմուլյացիայի արդյունքներ; dashboard-ը աշխատում է էմուլյատորով
13–18	Ու1. processing IP, AXI ինտեգրում, PS ծրագիր. Ու2. վիզուալիզացիա, վիճակագրություն	Տվյալները հարթակից հասնում են PS; փորձերի համեմատում
19–22	Ինտեգրում. FPGA ↔ սերվեր, runtime կոնֆիգուրացիա	Ամբողջական ցիկլը աշխատում է (բաժին 14, քայլեր 1–6)
23–25	Անոմալիաների հայտնաբերում, չափումներ (timing, ռեսուրսներ, latency)	Թեստերի և չափումների հաշվետվություն
26–28	Դիպլոմային աշխատանքների գրում	Ամբողջական սևագրեր ղեկավարներին
29–30	Պաշտպանության նախապատրաստում	Սլայդներ, ցուցադրման փորձ

Նշում

Հանդիպումներ՝ շաբաթը մեկ անգամ (ընդհանուր): Յուրաքանչյուր հանդիպումից առաջ ուսանողները ուղարկում են կարճ գրառում. ինչ է արված, ինչն է խոչընդոտում, հաջորդ քայլերը:

12. Արդյունքներ

Աղյուսակ 12.1: Նախագծի հանձնվող արդյունքները.

Արդյունք	Ձևաչափ	Ժամկետ	Պատասխանատու
RTL կոդ, testbench-եր, Vivado նախագիծ, PS ծրագիր	git ռեպոզիտորիա, README՝ կառուցման հրահանգներով	շաբ. 25	Ուսանող 1

Արդյունք	Չափ	Ժամկետ	Պատասխանատու
Backend, տվյալների բազայի սխեմա, Web հարթակ, էմուլյատոր	git ռեպոզիտորիա, README, տեղադրման հրահանգ	շաբ. 25	Ուսանող 2
Միջերեսի փաստաթուղթ	Markdown ռեպոզիտորիայում	շաբ. 5	Երկուսն էլ
Timing և ռեսուրսների հաշվետվություն	դիպլոմի բաժին + Vivado հաշվետվություններ	շաբ. 25	Ուսանող 1
Վերլուծության արդյունքներ (փորձերի համեմատություն, անոմալիաներ)	դիպլոմի բաժին + տվյալներ	շաբ. 25	Ուսանող 2
Դիպլոմային աշխատանք (յուրաքանչյուրը՝ առանձին)	PDF (diploma-project ձևանմուշ)	[ամսաթիվ]	Յուրաքանչյուրը
Ցուցադրում և սլայդներ	կենդանի ցուցադրում + տեսագրություն	[ամսաթիվ]	Երկուսն էլ

13. Գնահատում և ընդունման չափանիշներ

13.1 Ընդունման չափանիշներ

Նախագիծը ընդունվում է պաշտպանության, եթե.

- MVP-ն (բաժին 4) աշխատում է և ցուցադրվում է AUP ZU3-ի վրա;
- runtime կոնֆիգուրացիան Web հարթակից փոխում է FPGA-ի վարքը առանց նոր bitstream-ի;
- յուրաքանչյուր RTL մոդուլ ունի testbench, նախագիծը անցնում է timing-ը;
- կոդը և հրահանգները ռեպոզիտորիայում են, և մեկ այլ անձ կարող է կրկնել կառուցումը:

13.2 Գնահատման աղյուսակ

Աղյուսակ 13.1: Գնահատման չափանիշներ (յուրաքանչյուր ուսանողի համար՝ իր մասով).

Չափանիշ	Կշիռ	Նվազագույն (բավարար)	Լավ	Գերազանց
Տեխնիկական արդյունք	35%	MVP-ն աշխատում է	MVP + չափված բնութագրեր (throughput, latency)	MVP + ընդլայնում, մաքուր ճարտարապետություն

Չափանիշ	Կշիռ	Նվազագույն (բավարար)	Լավ	Գերազանց
Վերիֆիկացիա և թեստավորում	20%	Հիմնական մոդուլները թեստավորված են	Ինքնաստուգվող թեստեր բոլոր մոդուլների համար	Ավտոմատացված թեստեր, ծածկույթի վերլուծություն
Վերլուծություն և ընբռնում	15%	Բացատրում է իր որոշումները	Համեմատում է այլընտրանքները	FPGA և ծրագրային մշակման խորքային համեմատություն
Աշխատանքի գործընթաց	10%	Հանգրվանները մեծ մասամբ պահպանված են	Կանոնավոր commit-ներ և հաշվետվություններ	Ինքնուրույնություն, ակտիվ համագործակցություն
Փաստաթուղթ և պաշտպանություն	20%	Ամբողջական դիպլոմ	Հստակ կառուցվածք, նկարներ, չափումներ	Հրապարակման մակարդակի շարադրանք և ցուցադրում

Նշում

Աղյուսակը լրացնում է, բայց չի փոխարինում համալսարանի պաշտոնական գնահատման կանոնները:

14. Ամփոփ ցուցադրման սցենար

1. Օգտվողը բացում է Web հարթակը, ընտրում **Vibration** սենսորը և **Periodic** ազդանշանը:
2. Կոնֆիգուրացնում է sample rate, ամպլիտուդ, աղմուկ, threshold, filter, anomaly և սեղմում **Start Experiment**:
3. Backend-ը կոնֆիգուրացիան ուղարկում է AUP ZU3-ին. Zynq PS-ը գրում է այն AXI4-Lite ռեգիստրներում:
4. FPGA-ն գեներացնում է վիբրացիանման տվյալներ և կատարում իրական ժամանակի մշակում:
5. Տվյալները PS-ից Ethernet-ով հասնում են սերվեր և պահվում բազայում:
6. Dashboard-ը ցուցադրում է raw և processed ազդանշանները իրական ժամանակում:
7. FPGA-ն ներարկում է անոմալիա. threshold detector-ը և սերվերային մոդելը հայտնաբերում են այն:
8. Հարթակը համեմատում է FPGA-ի հայտնաբերումը և ծրագրային վերլուծությունը:
9. Օգտվողը փոխում է FPGA-ի պարամետրերը, և վարքը փոխվում է **runtime**-ում՝ առանց bitstream վերագեներացնելու:

15. Ընդլայնումներ (ընտրովի)

Ստորև բերվածները **պարտադիր չեն**. դրանք կարող են բարձրացնել գնահատականը կամ դառնալ մագիստրոսական թեզի հիմք.

- **AXI DMA / DDR ուղի** — բարձր տվյալահոսքի համար:
- **FPGA reconfiguration** — նախապես գեներացված bitstream-ների ընտրություն Web հարթակից:
- **Ֆիզիկական սենսորներ** — էմուլացված աղբյուրների փոխարեն կամ կողքին:
- **Ադապտիվ հետադարձ կապ (closed loop)** — սերվերի մոդելը ավտոմատ փոխում է FPGA-ի պարամետրերը:
- **Linux Zynq PS-ի վրա** — սերվերային բաղադրիչների մասնակի տեղափոխում:

16. Ղեկավարում, համագործակցություն և ռիսկեր

16.1 Ղեկավարում

- **Ղեկավար 1** (Միկրոէլեկտրոնիկա) — RTL, FPGA, վերիֆիկացիա:
- **Ղեկավար 2** (Ծրագրային ճարտարագիտություն) — Backend, տվյալներ, վերլուծություն:
- Ընդհանուր հանդիպում՝ շաբաթը մեկ անգամ, ինտեգրման փուլում՝ երկու անգամ:

16.2 Ռիսկեր

Աղյուսակ 16.1: Հիմնական ռիսկեր և կանխարգելում.

Ռիսկ	Ազդեցություն	Կանխարգելում
AXI/Zynq ինտեգրման բարդությունը	բարձր	Ներածական վարժություն 2, պատրաստի AXI IP շաբլոններ, ղեկավարի աջակցություն
Ուսանող 2-ը սպասում է ապարատին	միջին	Էմուլյատոր-սկրիպտ նույն JSON ֆորմատով (Խ2.3)
Timing-ը չի անցնում	միջին	Pipeline, ցածր տակտային հաճախություն MVP-ի համար
Հարթակը հասանելի չէ	ցածր	Սիմուլյացիա, ժամանակացույց լաբորատորիայում

Գրականություն և ընթերցանության ցանկ

Գրքեր

[B1] Sarah L. Harris and David Money Harris. *Digital Design and Computer Architecture: ARM Edition*. Morgan Kaufmann, 2016.

[B2] Pong P. Chu. *FPGA Prototyping by SystemVerilog Examples: Xilinx MicroBlaze MCS SoC Edition*. Wiley, 2018.

[B3] Steven W. Smith. *The Scientist and Engineer's Guide to Digital Signal Processing*. California Technical Publishing, 1997. URL: `https://www.dspguide.com`.

[B4] Louise H. Crockett, Ross A. Elliot, Martin A. Enderwitz, and Robert W. Stewart. *The Zynq Book: Embedded Processing with the ARM Cortex-A9 on the Xilinx Zynq-7000 All Programmable SoC*. Strathclyde Academic Media, 2014.

[B5] Martin Kleppmann. *Designing Data-Intensive Applications*. O'Reilly Media, 2017.

[B6] Wes McKinney. *Python for Data Analysis*. 3rd ed. O'Reilly Media, 2022.

Հոդվածներ

[A1] Varun Chandola, Arindam Banerjee, and Vipin Kumar. "Anomaly Detection: A Survey". In: *ACM Computing Surveys* 41.3 (2009).

Ձեռնարկներ, տեխնիկական փաստաթղթեր և ստանդարտներ

[U1] AMD. *Vivado Design Suite User Guide: Logic Simulation (UG900)*. Advanced Micro Devices.

[U2] AMD. *Zynq UltraScale+ Device Technical Reference Manual (UG1085)*. Advanced Micro Devices.

[U3] Arm. *AMBA AXI Protocol Specification (IHI 0022)*. Arm Limited.

[U4] AMD. *AXI4-Stream FIFO LogiCORE IP Product Guide (PG080)*. Advanced Micro Devices.

[U5] AMD. *Vivado Design Suite User Guide: Design Analysis and Closure Techniques (UG906)*. Advanced Micro Devices.

Համացանցային աղբյուրներ

[W1] FastAPI. *FastAPI Documentation — Tutorial / User Guide*. URL: `https://fastapi.tiangolo.com` (visited on 10/04/2026).

[W2] The PostgreSQL Global Development Group. *PostgreSQL Documentation — Tutorial*. URL: https://www.postgresql.org/docs/ (visited on 10/04/2026).

Հապավումների ցանկ

FPGA Field-Programmable Gate Array.

RTL Register-Transfer Level.

AXI Advanced eXtensible Interface.

PL Programmable Logic.

PS Processing System.

MPSoC Multiprocessor System-on-Chip.

LFSR Linear-Feedback Shift Register.

FIFO First-In First-Out.

BRAM Block RAM.

DMA Direct Memory Access.

DDR Double Data Rate memory.

IP Intellectual Property core.

MVP Minimum Viable Product.

API Application Programming Interface.

FIR Finite Impulse Response.

Բառարան

bitstream FPGA-ի կոնֆիգուրացիայի երկուական ֆայլ, որը ծրագրավորում է ծրագրավորվող տրամաբանությունը (PL).

runtime կոնֆիգուրացիա արդեն իրականացված ապարատի պարամետրերի փոփոխություն՝ առանց նոր bitstream գեներացնելու.

experiment (փորձ) տվյալների հավաքագրման մեկ սեանս՝ որոշակի կոնֆիգուրացիայով, սկիզբ և ավարտ ունեցող.

testbench սիմուլյացիոն միջավայր, որը մուտքային ազդանշաններ է տալիս RTL մոդուլին և ստուգում ելքերը.

Հաստատում

Սույն առաջադրանքը սահմանում է նախագծի շրջանակը: Նպատակների, շրջանակի կամ արդյունքների փոփոխությունները համաձայնեցվում են ղեկավար(ներ)ի հետ և գրանցվում որպես նոր տարբերակ:

Ղեկավար 1
[Ղեկավարի անուն, ազգանուն, աստիճան]

_____ ստորագրություն

_____ Ամսաթիվ

Ղեկավար 2
[Ղեկավարի անուն, ազգանուն, աստիճան]

_____ ստորագրություն

_____ Ամսաթիվ